Fertilizer Recommendation Module Presentation Guide

1. Objective of the Module

The goal of this module is to suggest the ideal fertilizer for a specific crop to improve soil health and maximize yield. Instead of relying on guesswork, the model predicts the exact fertilizer needed based on current soil nutrient levels and environmental factors.

2. Dataset Explanation (`fertilizer_recommend.csv`)

The dataset consists of **553 data records** representing various farming conditions.

Input Features (Independent Variables): There are 8 parameters that the farmer (or sensors) provides: * **Environmental Factors:** * `Temperature` (in Celsius) * `Humidity` (Relative humidity in %) * `Moisture` (Soil moisture level) * **Soil Nutrients (Macronutrients):** * `Nitrogen` (N) * `Phosphorous` (P) * `Potassium` (K) * **Categorical Factors:** * `Soil_Type` (Sandy, Loamy, Black, Red, Clayey) * `Crop_Type` (Wheat, Maize, Sugarcane, Cotton, Paddy, Barley, etc.)

Target Variable (Dependent Variable): * `Fertilizer`: The exact fertilizer recommended (e.g., Urea, DAP, 14-35-14, 28-28, 17-17-17, 20-20, 10-26-26).

3. Data Preprocessing in the `.ipynb` File

Before feeding the data to the machine learning models, the notebook performs critical preprocessing steps: 1. **Label Encoding:** Machine learning models only understand numbers. The categorical variables (`Soil_Type`, `Crop_Type`) and the target variable (`Fertilizer`) are converted into integer IDs using `LabelEncoder`. (e.g., Sandy = 0, Loamy = 1). 2. **Feature Scaling:** The numerical values (like Nitrogen which might be high, and Temperature which might be lower) are scaled down to a standardized range (usually 0 to 1) using a `MinMaxScaler`. This ensures that larger numerical values do not unfairly bias the model. 3. **Train-Test Split:** The dataset is split into training data (to teach the model) and testing data (to evaluate its performance on unseen data).

4. Graphs and Visualizations (Exploratory Data Analysis)

The notebook generates several important graphs to understand the data and model behavior. Here is how you explain them:

A. Feature Importance Graphs (Horizontal Bar Charts)

- **What is happening:** The notebook plots feature importances for both the Decision Tree and the Random Forest models.

- **How to explain it:** These graphs show us which input features have the most influence on deciding the fertilizer. For example, you will notice that macronutrients (Nitrogen, Phosphorous, Potassium) and Crop Type usually have the longest bars, meaning they are the heaviest deciding factors, while Temperature or Humidity might play a secondary role.

B. The Decision Tree Plot (`plot_tree`)

- **What is happening:** A visual flowchart showing the logic rules created by the Decision Tree algorithm.

- **How to explain it:** This graph visually represents the "If-Else" rules the model learned. For example, the top node might say "If Nitrogen <= X, go left, else go right." It shows exactly how the model splits the data down to the "leaves" (the final fertilizer prediction).

C. Cross-Validation Line Plot

- **What is happening:** A line chart showing `n_estimators` (number of trees) on the X-axis and `Mean CV Accuracy` on the Y-axis.

- **How to explain it:** This graph is used to "tune" the Random Forest model. A Random Forest is made up of many decision trees. This graph plots how accurate the model is when we use 10, 50, 100, 200, and 300 trees. We use this to find the "sweet spot" where the model is highly accurate but not overly complex (in this case, `n_estimators=100` was chosen).

5. Model Building and Comparison

The notebook doesn't just build one model; it tests several standard classification algorithms to find the best one: * Logistic Regression * Support Vector Classifier (SVC) * K-Nearest Neighbors (KNN) * Naive Bayes * Decision Tree * Random Forest

The Results: After generating a classification report and comparing the accuracy scores, the **Random Forest** and **KNN** models achieved the highest accuracy (nearly 100%).

6. The Final Selection and Export

- **Chosen Model:** The **Random Forest Classifier** is selected as the final model because it is highly robust against overfitting and handles tabular data extremely well. It was trained with a max depth of 4 and 100 estimators.

- **Deployment Integration:** The final step in the notebook uses a library called `joblib` to save the trained Random Forest model as a file named `Fertilizer_Recommend.pkl`. It also saves the scaling logic as `Fertilizer_scaler.pkl`.

- **How it connects to the project:** The Django backend loads these `.pkl` files. When a farmer submits the form on the website, Django scales their inputs using `Fertilizer_scaler.pkl`, feeds it to `Fertilizer_Recommend.pkl`, and returns the predicted fertilizer name to the screen.